

Trust but Verify

Validating Data Masking Failures

Siyuan Feng

Conf42 DevSecOps 2026
October 1, 2026

Hi, I'm Siyuan Feng

- Data analyst at a legal services company
- I work on Test Data Management (TDM): getting realistic data to DEV & QA teams without leaking anything
- Later get reassign as QA for software development
- Designed and ran the sensitive-column identification pipeline which we are going to talk about
- Author of dbmask, an open-source database masking tool (MIT)



dbmask

Discover and mask sensitive data in any database — open source, for everyone.

github.com/sealandseacat/dbmask

Based on my article: *“A Practical Pipeline for Identifying Sensitive Columns Before Test Data Masking”*

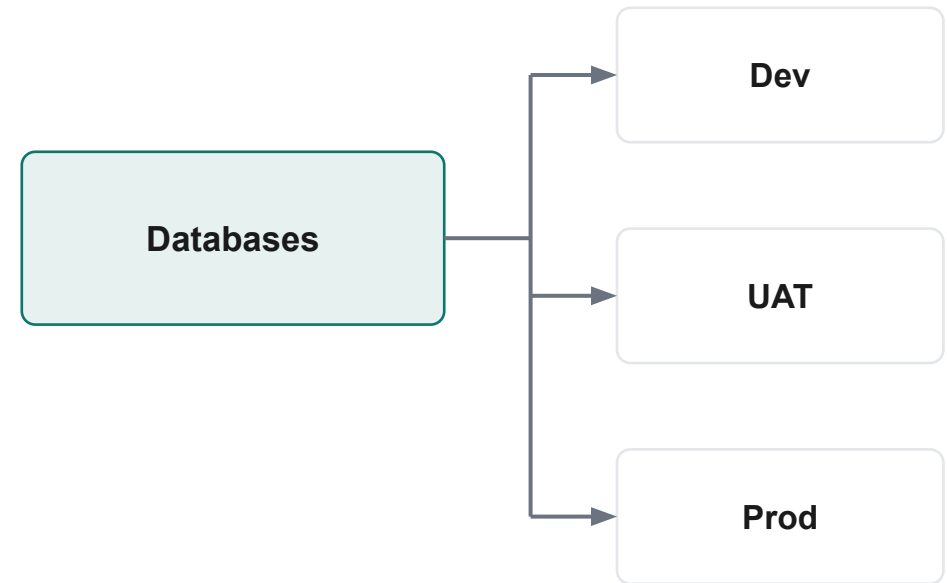
The next 30 minutes

- 1 Why we want to mask data
- 2 What is masking
- 3 How to decide successful masking

- 4 Steps of validation
- 5 dbmask: open source rebuild
- 6 Takeaways

Everyone needs test data. Everyone copies production data.

- Dev, QA and demo teams need realistic data — real **format** and real **volume**.
- The fastest way to get it: copy the production database
- Why? using different volume and format between Dev and Prod will make bugs and errors.
 - ex: your code may only be able to handle 100 rows in Dev instead 10000 rows in Prod
- Direct copy is most common approach to prepare test data



one copy per environment, refreshed on a cycle

Test environments don't have good protection

Production

- Strict access control
- Monitoring and audit trails
- Hardened perimeter(MFA; Firewall; strong password)

DEV/UAT

- Broader access: developers, contractors, vendors
- Weaker controls, lighter logging
- Lower cybersecurity requirements — by design, to save cost

Same customer data, weaker protection. In modern days, data leaks can cause serious damage financially, operationally and reputationally

So how to enhance protection? masking!

`everett@oldexample.com` → `bourrasque@newexample.com`

Replace real values with useable fakes, and keep consistent format

Raw Data Before and After Masking

Source:

	A	B	C	D	E	F	G
1	matter_id	contact_email	legacy_ref	notes	matter_title	email_template	intake_channel
2	1	m.okafor@northgate.example	412-88-3390	Documents received. Nothing outstanding.	Rivera v. Northgate LLC	welcome_v2	web
3	2	t.lindqvist@corp.example	233-14-7781	Client asked to be called back after 5pm.	Estate of M. Delgado	invoice_reminder	phone
4	3	a.bello@mesa-law.example	601-22-9043	Spouse Marisol Haddad is authorised to collect the file; her number is 415-555-0182.	Delgado v. City of Mesa	matter_closed	web
5	4	j.park@corp.example	155-90-2214	When the documents are delivered, ring the doorbell and call my wife Bobi.	In re: Bobi Tran	welcome_v2	referral

Target:

	A	B	C	D	E	F	G
	matter_id	contact_email	legacy_ref	notes	matter_title	email_template	intake_channel
	1	stephanie.flores881@example.in	828-55-4852		Mwcort g. Wlumxxahn EUH	welcome_v2	web
	2	jeffrey.adams724@example.invali	879-48-4789		Liaccy hz K. linsszm	invoice_reminder	phone
	3	mary.sanchez168@example.inva	897-54-2912		Yaotmqp m. Kaxe hw Lxkb	matter_closed	web
	4	donald.torres389@example.invali	022-43-7698		Ur au: Ldac Tbcl	welcome_v2	referral

how to define successful masking?

—a few observation from the previous slide

1. quantity of the data should stay the same: 7*5 tables after masking should stay 7*5
2. data format should stay the same: email address after masking is still email address
3. database elements should stay the same: primary key column stay primary key column
4. make sure data from each row is actually masked not skipped

Three questions before delivery

01 Row count

Do the number of rows stay same from source to target?

02 Elements

Do the required objects and rules work?

03 Data masking validation

Do sensitive values change after masking?

First, check that all expected rows arrived

Table	Before	After	Result
Customers	10,000	10,000	Match
Orders	30,000	29,800	200 missing

A filter, deletion or incomplete copy may be responsible.

Example contract: preserve every row in the approved source snapshot.

Compare every table, not just the total

Table	Before	After	Difference
Customers	10,000	10,100	+100
Orders	30,000	29,900	-100
TOTAL	40,000	40,000	0

Equal totals can hide two failures.

we also need to make sure the databases elements are matching between source and target environment:

Index, constraint, trigger, grant.

The customer who consume our masking data, usually also the developer of the application, and the owner of the source data. Their application may be able to run if it miss a index in target database

Index: a shortcut for finding records

Example: find customer C101 by their masked email.

Expected

Index on Customers.email

Useful for customer lookups

If the index is missing

A wider scan may be needed

Correct result, different performance

Check the definition and indexed columns — a matching name is not enough.

Constraint: a rule that protects valid data

Foreign key: every order must refer to an existing customer.

Order	customer_id	Customer exists?	Expected behavior
O501	C101	Yes	Accept
O503	C999	No	Reject

Missing or disabled FK → an orphan order may be accepted.

Check the rule, its enforcement state and the required relationship.

Trigger: an automatic database action

Example: inserting an order updates Customers.order_count.

Event for customer C101	Expected	Trigger missing
Before new order	order_count = 2	order_count = 2
After new order	order_count = 3	order_count = 2

Check the event, logic and required enabled state after masking.

Grant: who can do what

Example target policy: qa_reader can inspect test data.

Permission on Customers + Orders	Expected	Example failure
Read	Allowed	QA cannot open the data
Update / delete	Not allowed	QA can modify the data

Match the approved test-environment policy, including inherited permissions.

Data masking validation:

What if we miss masking some of the rows

It can happen when server run out of memory or when masking tool fail to transform some context with special sign or foreign language and other situation

how a missed masking row will look like

user_id	Source email	Target email
123ABC	everett@example.com	bourrasque@example.com
123ABC	思远@example.com	思远@example.com

思远@example.com retains the original email

Illustrative records. The key survives masking and uniquely identifies each row.

so how do we validate if data is truly masking across all rows

it depend on whether:

- 1, we have primary keys in source tables
- 2, we can form our own composite keys using non masking column
- 3, there is no way to form any kind of keys

With a stable key, match the same customer

customer_id	Email before	Email after
C101	everett@example.com	bourrasque@example.invalid
C102	pam@example.com	pam@example.com

C102 fails: the original email is still on the same record.

The comparison key must be stable and unique. Check each sensitive column.

No primary key? build your own composite keys!

Use stable, unmasked columns whose combination is unique.

batch_id	record_ref	Source name	Composite key
B1	01	Everett	(B1, 01)
B1	02	Pam	(B1, 02)
B2	01	Ben	(B2, 01)
B2	02	Bourrasque	(B2, 02)

but notice: Finding and building keys across huge tables takes time. and there is no guarantee you can find one

The most tricky situation: no primary key can be found and build in the table and there is no way to joint them to make comparison

cause for that can be:

Legacy tables and migrations

A declared, trustworthy key may be unavailable

Logs and duplicate records

Tables allow duplicate in structure on purpose

All column in the table need to be masked

No way to construct composite key yourself

The solution is to use intersect, not joint!

Source:			Target:	
customer name	password		customer name	company name
Super Pom	123ABC		Everette	111AAA
Super Pom	321ABC		Everette	222BBB
Benz	456EDF		Bourrasque	333CCC
Benz	654EDF		Bourrasque	123ABC

For this table, we replace all the password with randomly generated string value. in theory, there should be NO overlapping value between source password column and target password column

we only need intersect this two password column to find failed masking value. and in this case, the existence of overlapping value means there is a row failed masking.

Situation become much complicated when we use dictionary value to replace, and dictionary value already contain source data

in this case, simple intersection is not enough because overlapping value is legitimate to exist

Source:			Target:	
customer name	company name		customer name	company name
Super Pom	Apple		Super Pom	Oracle
Super Pom	Tesla		Super Pom	Microsoft
Benz	Google		Benz	Apple
Benz	Amazon		Benz	Meta

in this case, we use dictionary value to replace company name, and our dictionary already contain company name like Apple. so in theory, This can replace Apple with Oracle, which is good. This means data get masked. However, it may also replace Apple with Apple, which is bad, and our masking is missed. we just don't know which situation it is

The solution for dictionary-replacement:

intersect first to find overlapping value

then check rest of the non-masking column from source and target to make sure they are not the same

Source:			Target:	
customer name	company name		customer name	company name
Super Pom	Apple		Super Pom	Oracle
Super Pom	Tesla		Super Pom	Microsoft
Benz	Google		Benz	Apple
Benz	Amazon		Benz	Meta

In the case we had earlier: we had a Apple as overlapping common value, we should then check rest of the non masking column and see if they also the same, if they are not the same, in this case we get Super Pom vs Benz, it means no rows failed masking

dbmask — the same ideas, for everyone

“Discover and mask sensitive data in any database — open source, for everyone.”

- A clean-room rebuild — no company code, no proprietary rules
- Any Excel or SQL database via SQLAlchemy: PostgreSQL, MySQL / MariaDB, SQL Server, Oracle, SQLite...
- Python 3.9+, YAML configuration, MIT license
- v0.1.1 — an initial framework; contributions welcome

```
github.com/  
sealandseacat/dbmask
```

```
git clone → pip install -e .
```

dbmask validation report

100 synthetic customers, 10 columns, 8 sensitive columns. PostgreSQL via PGLite.

dbmask 0.1.1 Commit 516f34fa1777 Run 2026-09-14 05:03 UTC

1. Row counts

Table	Source	Target	Status	Checked by
Customers	100	100	PASS	dbmask
Orders	300	300	PASS	dbmask

2. Database elements

Element	Source	Target	Status	Checked by
Index	5	5	PASS	dbmask + catalog audit
Constraint	20	20	PASS	dbmask + catalog audit
Trigger	1	1	PASS	Demo supplement
Grant	2	2	PASS	Demo supplement

Grant scope: QA read-only on both tables. Constraint count includes NOT NULL.

3. Masking completeness

Check	Required	Verified	Status	Method
Rows with all 8 values changed	100	100	PASS	Every row, matched by PK
Sensitive values changed	800	800	PASS	8 columns x 100 rows
Original values retained	0	0	PASS	No unchanged sensitive values
Target format checks	800	800	PASS	Independent demo checks

Overall result

PASS

Full row coverage. Test-data skipping disabled. Trigger/grant checks are demo supplements.

Thank you

If you have any question, please feel free to reach out to me

Siyuan Feng · sealseacat@gmail.com

Code: github.com/sealandseacat/dbmask